

Before You Start

1 Binaries	Download the pre-built spiritbun TurboQuant package or use the bundled Update Binaries button. It contains llama-server.exe, llama-bench.exe, llama-cli.exe and all ggml-*.dll / llama.dll files.
2 NVIDIA GPU	Install CUDA Toolkit 12.x or 13.x (nvidia.com/cuda). Copy the matching cuBLAS DLLs (cublas64_12.dll + cublasLt64_12.dll for CUDA 12, or cublas64_13.dll + cublasLt64_13.dll for CUDA 13) into the llama-server folder. These DLLs are not bundled.
3 AMD GPU	Install ROCm 6.x (rocm.docs.amd.com). Use the HIP/ROCm build variant of llama-server. No extra DLLs needed — ROCm runtime ships its own libraries.
4 macOS (Apple Silicon)	Metal is built-in — no extra drivers or DLLs required. Use the macOS build variant. Works on M1 / M2 / M3 / M4.
5 First model	Download a GGUF model from Hugging Face. Good starter: meta-llama-3.1-8b-instruct-q4_k_m.gguf (~4.6 GB). Bigger / better: Qwen3.5-9B-UD-Q4_K_XL.gguf (~5.6 GB). Place the file anywhere, then add the folder in QLauncher → Paths.



CUDA version compatibility

CUDA 12.4–12.8: fully tested, recommended. CUDA 12.8+ required for RTX 5090 (sm_120). CUDA 13.2: experimental, may work. CUDA 13.1: DO NOT USE — known MMQ kernel segfault that causes crashes during inference. If you need CUDA 13.x, use 13.2 or newer.



Where to find the CUDA DLLs (Windows)

Start → search for "NVIDIA Control Panel" → Help → System Information shows your driver version. Then download the matching CUDA Toolkit. The two required DLLs are in the bin\ subfolder. Alternatively, copy them from any machine that already has CUDA 12.x installed.

What is TurboQuant QLauncher?

TurboQuant QLauncher is a lightweight desktop GUI for managing llama-server with TurboQuant KV-Cache compression. Launch large language models on any NVIDIA, AMD, Intel or Apple GPU with a single double-click, switch between models and binary builds instantly, run benchmark matrices, and monitor VRAM and system-RAM usage — all without touching the command line.



Why TurboQuant?

Standard llama.cpp stores the KV-cache in full float16 precision. TurboQuant compresses it using Walsh-Hadamard rotation (ICLR 2026, Google Research), achieving 4–5× smaller KV-cache with minimal quality loss. This means longer context, more VRAM headroom — or fitting a bigger model on the same hardware.

Key Features

 One-click Launch	Double-click any model row to start llama-server on that GPU. Double-click again to stop.
 Live VRAM Bars	Braille-style bars show effective VRAM usage per GPU and CPU RAM — updated live when you switch KV config or LA mode.
 KV-aware Colors	Green = fits comfortably. Amber = tight. Red = overflow. Bars and ✓ / ✗ markers update with the compression selection.
 Benchmarking	Built-in llama-bench: single config or Bench All matrix (KV × LA in one run). f16 baseline + deltas, saved as Markdown.
 Idle / Loading / Running	Status pill in the header shows ● Idle, ⏳ Loading while the model loads into VRAM, then ● Running (click to open the browser).
 Persistent Config	Window position, splitter, KV selection, port, ctx, timeout, model paths, and slot bookmarks are saved automatically on exit.
 Multi-Directory Scan	NEW v0.47 — Scan up to 3 separate model directories together. Optional recursive subdirectory walk. Duplicates are detected automatically.
 Quick-Switch Slots	v0.44 — Bookmark up to 6 llama-server.exe builds (mainline / TheTom / spiritbuun / Gemma4 / etc.). One click in the footer switches the active build.
 Layer-Adaptive	Keep critical layers (first / last) at q8_0 for best quality. Modes 1, 5, 7 via LA buttons, passed as TURBO_LAYER_ADAPTIVE env var.
 Bench All Matrix	Matrix benchmark: every selected KV config × every selected LA mode in one run. Stoppable between runs. Per-run timeout configurable.
 Universal GPU Detection	NVIDIA (nvidia-smi), AMD (rocm-smi), Intel and Apple Silicon (WMI / lspci / sysctl). Mixed-architecture systems are supported.

Header Controls

KV row	Six KV-cache modes: f16, q8 ₀ +t4, t3/t3, t4/t4, q8 ₀ +t3, q8 ₀ /q8 ₀ . Click to select; live VRAM bars and fit indicators update immediately.
Port	TCP port for llama-server (default 8080). Change here before launch.
Ctx	v0.42 — Context window in tokens. Empty = use the model's default. Any value is passed to llama-server as -c <ctx>.
Timeout	v0.46 — Per-run timeout for llama-bench in seconds (default 90). Prevents hangs in Bench All matrices.
☐ No Thinking	Disable reasoning / thinking mode via --reasoning off. Useful for tool-calling models.
☐ Benchmark	Single-config benchmark mode. Double-click or Run executes f16 baseline + the currently selected KV config and writes a delta-% table.
☐ Bench All	Activates the full KV × LA matrix. All KV and LA buttons turn blue; click any to deselect. Run starts, Stop cancels after the current run.
LA: off / 1 / 5 / 7	Layer-Adaptive modes: off (uniform), 1 (boundary-V), 5 (first4 + last4 at q8_0), 7 (aggressive). In Bench All they toggle individually.

Run / Stop	Run starts the benchmark with the current selection. Stop cancels after the in-flight run completes.
● Status pill	Idle / Loading / Running. Click while Running to open http://localhost:{port} in the browser.

Footer Controls

Rescan Models	Rescan all configured model directories now. Honours the recursive flag.
Update Binaries	Run the bundled binary updater for the active llama-server folder.
Slot buttons	v0.44 — One-click switch between bookmarked llama-server.exe builds. The active slot is highlighted. Right-click in the Paths dialog to edit.
 Save Results...	Turns green when a benchmark finishes. Click to append the result table to TurboQuant_Benchmark.md next to the launcher.
Paths	Open the Configure Paths dialog: model directories, recursive flag, active llama-server.exe, and quick-switch slots.
About	Version, license, build info, and links to upstream projects.

Mouse & Keyboard

Double-click row	Launch server with the selected model on that GPU (or stop if already running).
Single click	Select a model / GPU row without launching.
↑ ↓ / ← →	Navigate models (up / down) and GPUs within a model (left / right).
Enter	Toggle the server for the currently selected row.
Escape	Stop the running server.
Click status pill	Open http://localhost:{port} in the browser (only when Running).
Drag splitter	Resize model list vs. log area.

Configure Paths Dialog

Open via the Paths button in the footer. The dialog has three sections.

LLM Models Directories (NEW v0.47 — up to 3)

Three numbered slots (#1, #2, #3) for model directories. Each row has a text field and a browse button. Leave a slot empty to disable it. All non-empty slots are scanned together and the results are merged. Duplicate files (same realpath, e.g. via symlinks or overlapping directories) are detected and listed only once.

- **Include subdirectories** — Recursive scan checkbox below the three rows. When checked (default), every subdirectory of each path is walked via `os.walk`. When unchecked, only the top level of each directory is scanned via `os.listdir`.
- **Backward compatible** — Existing v0.46 configs with a single `llm_models_path` are migrated automatically into slot #1 on first start. The legacy key is removed from the config file.

llama-server.exe Path (active)

The currently active llama-server.exe. This is the binary that Run, double-click, and Bench All actually invoke. Switch quickly via the slot buttons in the footer, or browse manually here.

Quick-Switch Slots (v0.44 — up to 6)

Bookmark additional llama-server.exe builds. Each row shows a derived label (auto-extracted from the parent folder name) on the left, the full path in the middle, and a browse button on the right. Saved slots appear as one-click buttons in the footer next to Update Binaries.



Typical multi-build setup

Slot 1: mainline_cuda132 (upstream baseline). Slot 2: thetom_cuda128 (TheTom turboquant_plus, CUDA 12.8 for RTX 5090). Slot 3: thetom_cuda132 (same fork, CUDA 13.2). Slot 4: gemma4_cuda132 (Gemma-4 specialised). One click switches the entire benchmark stack.

KV-Cache Configurations

f16	Baseline. Full float16 KV-cache, no compression. Used as reference for benchmark deltas.
q8 _o +t4	K at q8_0, V at turbo4. Best general-purpose compression. ≈3.5× smaller KV-cache, near-zero quality loss.
t3/t3	Both K and V at turbo3. Most aggressive that still preserves quality on most models. ≈4.5× smaller.
t4/t4	Both K and V at turbo4. Maximum compression. Some models show measurable quality drop — verify with a benchmark.
q8 _o +t3	K at q8_0, V at turbo3. Conservative middle ground.
q8 _o /q8 _o	Both K and V at q8_0. The classic non-TurboQuant 8-bit baseline for comparison.



Tip

The VRAM bars and ✓ / ✗ indicators update live as you click KV buttons. Watch a model that shows amber on your GPU turn green when you select t3/t3 or q8₀+t4!

Benchmarking & Documentation

QLauncher integrates llama-bench directly, so you can systematically benchmark every KV-Cache configuration without leaving the GUI. Results are saved as Markdown tables — ready for GitHub posts, internal documentation, or comparison across builds and hardware.

☐ Benchmark	Single-config mode. Select a model + GPU row, check Benchmark, then double-click, Enter, or Run. Runs f16 baseline + the currently selected KV config. Delta (%) is calculated automatically.
☐ Bench All	Full-matrix mode. Check Bench All → all KV + LA buttons turn blue. Click individual buttons to deselect configs you don't need. Press Run to start the KV × LA matrix (e.g. 6 KV × 4 LA = 24 runs). Stop cancels after the current run. Each run is bounded by the Timeout field (v0.46).
 Save Results	After a benchmark completes, the Save Results button turns green. Click to append the results to TurboQuant_Benchmark.md next to the launcher. Each run is a timestamped Markdown table with model name, GPU, KV config, and LA mode.
Benchmark.md	The file accumulates results over time — building a persistent log across CUDA versions, builds, models, and GPUs. Ideal for tracking regressions, comparing spiritbuun vs. TheTom builds, or preparing community posts.



Testing workflow for new builds

1) Build new binaries → bookmark them in a Quick-Switch slot. 2) Click the slot to make them active. 3) Select your reference model (e.g. Llama 3.1 8B Q4_K_M). 4) Check Bench All, deselect LA modes you don't need, set a sensible Timeout. 5) Press Run → wait. 6)  Save Results. 7) Switch slot, repeat. The resulting Markdown tables are ready to paste into Discussion #20969.

Open Source Projects & Acknowledgements

★ spiritbuun / llama.cpp	TurboQuant CUDA implementation · this build · github.com/spiritbuun/llama-cpp-turboquant-cuda
★ TheTom / turboquant_plus	Alternative TurboQuant build with fused mmvq kernel · github.com/TheTom/turboquant_plus
★ ggml-org / llama.cpp	Upstream llama.cpp project · github.com/ggml-org/llama.cpp
💬 Community Discussion	Benchmark findings · seanrasch · AmesianX · primoco et al. · github.com/ggml-org/llama.cpp/discussions/20969
 TurboQuant paper Zandieh et al., "TurboQuant: KV Cache Compression via Walsh–Hadamard Rotation", ICLR 2026, Google Research. This tool is an independent community project, not affiliated with Google.	

Voraussetzungen

1 Binaries	Lade das vorkompilierte spiritbuun-TurboQuant-Paket herunter oder verwende den eingebauten Update-Binaries-Button. Es enthält llama-server.exe, llama-bench.exe, llama-cli.exe und alle ggml-*.dll- / llama.dll-Dateien.
2 NVIDIA-GPU	Installiere CUDA Toolkit 12.x oder 13.x (nvidia.com/cuda). Kopiere die passenden cuBLAS-DLLs (cublas64_12.dll + cublasLt64_12.dll für CUDA 12 bzw. cublas64_13.dll + cublasLt64_13.dll für CUDA 13) in den llama-server-Ordner. Diese DLLs sind nicht im Paket enthalten.
3 AMD-GPU	Installiere ROCm 6.x (rocm.docs.amd.com). Verwende die HIP/ROCm-Build-Variante von llama-server. Keine zusätzlichen DLLs nötig — die ROCm-Runtime liefert eigene Bibliotheken mit.
4 macOS (Apple Silicon)	Metal ist eingebaut — keine zusätzlichen Treiber oder DLLs erforderlich. Verwende die macOS-Build-Variante. Läuft auf M1 / M2 / M3 / M4.
5 Erstes Modell	Lade ein GGUF-Modell von Hugging Face. Guter Einstieg: meta-llama-3.1-8b-instruct-q4_k_m.gguf (~4,6 GB). Größer / besser: Qwen3.5-9B-UD-Q4_K_XL.gguf (~5,6 GB). Datei beliebig ablegen, Ordner dann in QLauncher → Paths eintragen.

⚠ CUDA-Versionskompatibilität

CUDA 12.4–12.8: vollständig getestet, empfohlen. CUDA 12.8+ erforderlich für RTX 5090 (sm_120). CUDA 13.2: experimentell, kann funktionieren. CUDA 13.1: NICHT VERWENDEN — bekannter MMQ-Kernel-Segfault, der Abstürze während der Inferenz verursacht. Falls CUDA 13.x benötigt wird, mindestens 13.2 oder neuer einsetzen.

💡 Wo finde ich die CUDA-DLLs? (Windows)

Start → „NVIDIA Control Panel“ suchen → Hilfe → Systeminformationen zeigt die Treiberversion. Passenden CUDA Toolkit herunterladen. Die zwei benötigten DLLs liegen im Unterordner bin\. Alternativ: von einem Rechner kopieren, auf dem CUDA 12.x bereits installiert ist.

Was ist TurboQuant QLauncher?

TurboQuant QLauncher ist eine schlanke Desktop-GUI zum Verwalten von llama-server mit TurboQuant-KV-Cache-Komprimierung. Per Doppelklick startest du Sprachmodelle auf einer beliebigen NVIDIA-, AMD-, Intel- oder Apple-GPU, wechselst blitzschnell zwischen Modellen und Binary-Builds, führst Benchmark-Matrizen durch und überwachst VRAM- und System-RAM-Verbrauch — alles ohne Kommandozeile.

💡 Warum TurboQuant?

Standard-llama.cpp speichert den KV-Cache in voller float16-Genauigkeit. TurboQuant komprimiert ihn mittels Walsh-Hadamard-Rotation (ICLR 2026, Google Research) auf 4–5× kleinere Größe — bei minimalem

Qualitätsverlust. Das Ergebnis: längere Kontextfenster, mehr VRAM-Spielraum oder Platz für größere Modelle auf derselben Hardware.

Hauptfunktionen

 Ein-Klick-Start	Doppelklick auf eine Modellzeile startet llama-server auf dieser GPU. Nochmal doppelklicken stoppt.
 Live-VRAM-Balken	Braille-Balken zeigen den effektiven VRAM-Verbrauch pro GPU sowie CPU-RAM — werden live aktualisiert beim KV- oder LA-Wechsel.
 KV-bewusste Farben	Grün = passt bequem. Gelb = knapp. Rot = Überlauf. Balken und ✓ / ✗ ändern sich mit der Kompressionsauswahl.
 Benchmarking	Integriertes llama-bench: Einzelconfig oder Bench-All-Matrix (KV × LA in einem Lauf). f16-Baseline + Deltas, gespeichert als Markdown.
 Idle / Laden / Laufend	Status-Pille im Header zeigt ● Idle, ⏳ Laden während das Modell in den VRAM geladen wird, dann ● Laufend (Klick öffnet Browser).
 Persistente Config	Fensterposition, Splitter, KV-Auswahl, Port, Ctx, Timeout, Modell-Pfade und Slot-Lesezeichen werden beim Schließen automatisch gespeichert.
 Mehrere Modell-Verzeichnisse	NEU v0.47 — Bis zu 3 separate Modell-Verzeichnisse gleichzeitig scannen. Optionales rekursives Durchsuchen der Unterordner. Dubletten werden automatisch erkannt.
 Quick-Switch-Slots	v0.44 — Bis zu 6 llama-server.exe-Builds als Lesezeichen (mainline / TheTom / spiritbuun / Gemma4 / etc.). Ein Klick im Footer wechselt den aktiven Build.
 Layer-Adaptive	Kritische Layer (erste / letzte) bei q8_0 halten für beste Qualität. Modi 1, 5, 7 über LA-Buttons, wird als TURBO_LAYER_ADAPTIVE-ENV-Variable gesetzt.
 Bench-All-Matrix	Matrix-Benchmark: jede gewählte KV-Config × jeder gewählte LA-Modus in einem Lauf. Zwischen Runs stoppbar. Per-Run-Timeout konfigurierbar.
 Universelle GPU-Erkennung	NVIDIA (nvidia-smi), AMD (rocm-smi), Intel und Apple Silicon (WMI / lspci / sysctl). Mixed-Architecture-Systeme werden unterstützt.

Header-Steuerelemente

KV-Reihe	Sechs KV-Cache-Modi: f16, q8_0+t4, t3/t3, t4/t4, q8_0+t3, q8_0/q8_0. Klick zum Auswählen; VRAM-Balken und Fit-Anzeigen aktualisieren sich sofort.
Port	TCP-Port für llama-server (Standard 8080). Vor dem Start hier ändern.
Ctx	v0.42 — Kontextfenster in Tokens. Leer = Modell-Standard verwenden. Jeder Wert wird als -c <ctx> an llama-server übergeben.
Timeout	v0.46 — Timeout pro Lauf für llama-bench in Sekunden (Standard 90). Verhindert Hänger in Bench-All-Matrizen.
☐ No Thinking	Reasoning- / Thinking-Modus deaktivieren via --reasoning off. Nützlich für Tool-Calling-Modelle.
☐ Benchmark	Einzelkonfigurations-Modus. Doppelklick oder Run führt f16-Baseline + die aktuell gewählte

	KV-Config aus und schreibt eine Delta-%-Tabelle.
☐ Bench All	Aktiviert die volle KV × LA-Matrix. Alle KV- und LA-Buttons werden blau; Klick zum Abwählen einzelner. Run startet, Stop bricht nach dem aktuellen Lauf ab.
LA: off / 1 / 5 / 7	Layer-Adaptive-Modi: off (uniform), 1 (Boundary-V), 5 (erste4 + letzte4 bei q8_0), 7 (aggressiv). Bei Bench All einzeln togglebar.
Run / Stop	Run startet den Benchmark mit der aktuellen Auswahl. Stop bricht nach Abschluss des laufenden Runs ab.
● Status-Pille	Idle / Laden / Laufend. Klick während Laufend öffnet http://localhost:{port} im Browser.

Footer-Steuerelemente

Rescan Models	Alle konfigurierten Modell-Verzeichnisse jetzt neu scannen. Berücksichtigt das Recursive-Flag.
Update Binaries	Den eingebauten Binary-Updater für den aktiven llama-server-Ordner ausführen.
Slot-Buttons	v0.44 — Ein-Klick-Wechsel zwischen Lesezeichen für llama-server.exe-Builds. Der aktive Slot ist hervorgehoben. Im Paths-Dialog editieren.
 Save Results...	Wird grün, sobald ein Benchmark fertig ist. Klick hängt die Ergebnistabelle an TurboQuant_Benchmark.md neben dem Launcher an.
Paths	Öffnet den Configure-Paths-Dialog: Modell-Verzeichnisse, Recursive-Flag, aktive llama-server.exe und Quick-Switch-Slots.
About	Version, Lizenz, Build-Info und Links zu den Upstream-Projekten.

Maus & Tastatur

Doppelklick Zeile	Server mit ausgewähltem Modell auf dieser GPU starten (oder stoppen, wenn bereits aktiv).
Einfachklick	Modell- / GPU-Zeile auswählen, ohne zu starten.
↑ ↓ / ← →	Modelle navigieren (hoch / runter) und GPUs innerhalb eines Modells (links / rechts).
Enter	Server für aktuell ausgewählte Zeile umschalten.
Escape	Laufenden Server stoppen.
Klick Status-Pille	http://localhost:{port} im Browser öffnen (nur wenn Laufend).
Splitter ziehen	Modellliste vs. Log-Bereich vergrößern / verkleinern.

Configure-Paths-Dialog

Über den Paths-Button im Footer öffnen. Der Dialog hat drei Bereiche.

LLM Models Directories (NEU v0.47 — bis zu 3)

Drei nummerierte Slots (#1, #2, #3) für Modell-Verzeichnisse. Jede Zeile hat ein Textfeld und einen Browse-Button. Leere Slots werden ignoriert. Alle nicht-leeren Slots werden gemeinsam gescannt und die Ergebnisse zusammengeführt. Doppelte Dateien (gleicher Realpath, z.B. über Symlinks oder überlappende Verzeichnisse) werden erkannt und nur einmal gelistet.

- **Unterverzeichnisse einbeziehen** — Recursive-Scan-Checkbox unter den drei Zeilen. Wenn aktiviert (Standard), wird jedes Unterverzeichnis jedes Pfades per `os.walk` durchlaufen. Wenn deaktiviert, wird nur die oberste Ebene jedes Verzeichnisses per `os.listdir` gescannt.
- **Rückwärtskompatibel** — Bestehende v0.46-Configs mit einem einzelnen `llm_models_path` werden beim ersten Start automatisch in Slot #1 migriert. Der Legacy-Key wird aus der Config-Datei entfernt.

llama-server.exe Path (active)

Die aktuell aktive llama-server.exe. Das ist die Binary, die Run, Doppelklick und Bench All tatsächlich aufrufen. Schneller Wechsel über die Slot-Buttons im Footer oder manuell hier browsen.

Quick-Switch Slots (v0.44 — bis zu 6)

Lesezeichen für zusätzliche llama-server.exe-Builds. Jede Zeile zeigt links ein abgeleitetes Label (automatisch aus dem übergeordneten Ordernamen extrahiert), in der Mitte den vollen Pfad und rechts einen Browse-Button. Gespeicherte Slots erscheinen als Ein-Klick-Buttons im Footer neben Update Binaries.



Typisches Multi-Build-Setup

Slot 1: `mainline_cuda132` (Upstream-Baseline). Slot 2: `thetom_cuda128` (TheTom turboquant_plus, CUDA 12.8 für RTX 5090). Slot 3: `thetom_cuda132` (gleicher Fork, CUDA 13.2). Slot 4: `gemma4_cuda132` (Gemma-4-spezialisiert). Ein Klick wechselt den kompletten Benchmark-Stack.

KV-Cache-Konfigurationen

f16	Baseline. Voller float16-KV-Cache, keine Komprimierung. Dient als Referenz für Benchmark-Deltas.
q8 ₀ +t4	K bei q8 ₀ , V bei turbo4. Beste Allzweck-Komprimierung. ≈3,5× kleinerer KV-Cache, nahezu kein Qualitätsverlust.
t3/t3	Sowohl K als auch V bei turbo3. Aggressivste Variante, die bei den meisten Modellen die Qualität noch erhält. ≈4,5× kleiner.
t4/t4	Sowohl K als auch V bei turbo4. Maximale Komprimierung. Manche Modelle zeigen messbaren Qualitätsverlust — mit Benchmark verifizieren.
q8 ₀ +t3	K bei q8 ₀ , V bei turbo3. Konservativer Mittelweg.
q8 ₀ /q8 ₀	Sowohl K als auch V bei q8 ₀ . Die klassische Nicht-TurboQuant-8-Bit-Baseline zum Vergleich.

Tipp

Die VRAM-Balken und ✓ / ✗-Anzeigen aktualisieren sich live beim Klick auf KV-Buttons. Beobachte, wie ein Modell, das auf deiner GPU Gelb anzeigt, beim Wechsel auf t3/t3 oder q8₀+t4 Grün wird!

Benchmarking & Dokumentation

QLauncher integriert llama-bench direkt, sodass alle KV-Cache-Konfigurationen systematisch getestet werden können — ohne die GUI zu verlassen. Ergebnisse werden als Markdown-Tabellen gespeichert, fertig für GitHub-Posts, interne Dokumentation oder Vergleiche zwischen Builds und Hardware.

☐ Benchmark	Einzelkonfiguration. Modell + GPU-Zeile wählen, Benchmark ankreuzen, dann Doppelklick, Enter oder Run. Führt f16-Baseline + die aktuell gewählte KV-Config aus. Delta (%) wird automatisch berechnet.
☐ Bench All	Vollmatrix-Modus. Bench All ankreuzen → alle KV- + LA-Buttons werden blau. Einzelne Buttons klicken zum Abwählen. Run drücken startet KV × LA-Matrix (z.B. 6 KV × 4 LA = 24 Runs). Stop bricht nach dem aktuellen Run ab. Jeder Run wird durch das Timeout-Feld begrenzt (v0.46).
 Save Results	Nach Abschluss wird der Save-Results-Button grün. Klick hängt die Ergebnisse an TurboQuant_Benchmark.md neben dem Launcher an. Jeder Run ist eine Markdown-Tabelle mit Zeitstempel, Modellname, GPU, KV-Config und LA-Modus.
Benchmark.md	Die Datei sammelt Ergebnisse über die Zeit — ein persistentes Log über CUDA-Versionen, Builds, Modelle und GPUs. Ideal für Regressionstests, Vergleich spiritbuun vs. TheTom, oder Vorbereitung von Community-Posts.

Test-Workflow für neue Builds

1) Neue Binaries bauen → in einem Quick-Switch-Slot bookmarken. 2) Slot anklicken um sie zu aktivieren. 3) Referenzmodell wählen (z.B. Llama 3.1 8B Q4_K_M). 4) Bench All ankreuzen, unnötige LA-Modi abwählen, sinnvolles Timeout setzen. 5) Run drücken → warten. 6)  Save Results. 7) Slot wechseln, wiederholen. Die Markdown-Tabellen sind sofort bereit für Discussion #20969.

Open-Source-Projekte & Danksagungen

★ spiritbuun / llama.cpp	TurboQuant-CUDA-Implementierung · dieser Build · github.com/spiritbuun/llama-cpp-turboquant-cuda
★ TheTom / turboquant_plus	Alternativer TurboQuant-Build mit fused mmvq-Kernel · github.com/TheTom/turboquant_plus
★ ggml-org / llama.cpp	Upstream-llama.cpp-Projekt · github.com/ggml-org/llama.cpp
💬 Community-Diskussion	Benchmark-Ergebnisse · seanrasch · AmesianX · primoco et al. · github.com/ggml-org/llama.cpp/discussions/20969
 TurboQuant-Paper	Zandieh et al., „TurboQuant: KV Cache Compression via Walsh–Hadamard Rotation“, ICLR 2026, Google Research. Dieses Tool ist ein unabhängiges Community-Projekt und nicht mit Google verbunden.